

AutoML

Автоматизация машинного обучения: теория и практика

Python · TPOT · Auto-sklearn · H2O AutoML · Optuna · SHAP

Тема 1. Введение в AutoML: мотивация и определение

AutoML (Automated Machine Learning) — это направление в области машинного обучения, целью которого является автоматизация полного цикла построения предиктивных моделей: от предобработки исходных данных до развёртывания обученной модели в продуктивной среде. Необходимость в AutoML обусловлена экспоненциальным ростом объёма данных и одновременным дефицитом квалифицированных специалистов в области Data Science.

Традиционный подход к построению ML-модели предполагает последовательное выполнение этапов: очистка данных, конструирование признаков (feature engineering), выбор алгоритма, настройка гиперпараметров и валидация результата. Каждый из этих этапов требует экспертных знаний и значительных временных затрат. AutoML формализует данный процесс как задачу оптимизации: пространством поиска является множество всех допустимых конфигураций пайплайна, а целевой функцией — метрика качества на валидационной выборке.

Ключевая теоретическая задача AutoML известна как CASH (Combined Algorithm Selection and Hyperparameter optimization). Формально, для набора алгоритмов $A = \{A_1, \dots, A_k\}$ с пространствами гиперпараметров $\Lambda_1, \dots, \Lambda_k$ требуется найти пару (A^*, λ^*) , минимизирующую ошибку на кросс-валидации. Это NP-трудная задача, решаемая эвристическими методами: байесовской оптимизацией, генетическими алгоритмами и методами бандитов.

Тема 1. Экосистема AutoML-библиотек

Библиотека	Метод поиска	Ключевые особенности
TPOT	Генетическое программирование	Экспорт пайплайна в Python-код
Auto-sklearn	Байесовская оптимизация (SMAC)	Мета-обучение, ансамблирование
H2O AutoML	Перебор + stacking	Масштабируемость, leaderboard
Optuna	TPE (Tree Parzen Estimator)	Гибкий API, pruning, визуализация
AutoGluon	Stacking + bagging	Таблицы, текст, изображения
FLAML	CFO + BlendSearch	Экономичный поиск (Microsoft)

```
# Установка ключевых AutoML-библиотек
!pip install tpot auto-sklearn h2o optuna autogluon

# Базовые импорты для всех экспериментов
import numpy as np, pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, f1_score
```

Тема 1. Сравнение стратегий поиска

Все AutoML-системы решают одну и ту же задачу — поиск оптимальной конфигурации в комбинаторном пространстве пайплайнов, но применяют принципиально различные стратегии.

Генетическое программирование (TPOT)

Кодирует пайплайн как дерево и применяет эволюционные операторы — селекцию, скрещивание, мутацию.

Преимущество: способность обнаруживать нетривиальные структуры пайплайнов.

Недостаток: высокая вычислительная стоимость, стохастичность результатов.

Байесовская оптимизация (Auto-sklearn, Optuna)

Строит суррогатную модель целевой функции (Gaussian Process или TPE). На каждой итерации выбирается точка, максимизирующая функцию приобретения (Expected Improvement, Upper Confidence Bound).

Преимущество: эффективность при малом бюджете вычислений.

Недостаток: сложность работы с условными пространствами.

Полный перебор с ансамблированием (H2O)

Обучает фиксированный набор алгоритмов с предопределёнными сетками гиперпараметров и комбинирует лучшие модели через Stacked Ensemble.

Преимущество: простота, детерминированность, масштабируемость.

Недостаток: ограниченное пространство поиска.

Тема 2. Архитектура AutoML-пайплайна

ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

AutoML-пайплайн представляет собой ориентированный ациклический граф (DAG), в котором каждый узел соответствует оператору преобразования данных или алгоритму обучения. Структура пайплайна определяется тремя уровнями автоматизации.

Первый уровень — предобработка данных (**data preprocessing**)

Сюда входят: обработка пропущенных значений (импутация средним, медианой, KNN-импутация), кодирование категориальных признаков (One-Hot, Target, Ordinal Encoding), масштабирование числовых признаков (StandardScaler, MinMaxScaler, RobustScaler).

Второй уровень — конструирование и отбор признаков (**feature engineering / selection**)

Автоматическое конструирование включает полиномиальные взаимодействия, агрегатные статистики и PCA/t-SNE для снижения размерности. Отбор признаков реализуется через фильтрующие методы (mutual information, chi-squared), обёрточные методы (RFE) и встроенные методы (L1-регуляризация, feature importance).

Третий уровень — выбор модели и настройка гиперпараметров (**CASH**)

AutoML-система перебирает множество алгоритмов и оптимизирует их гиперпараметры с использованием кросс-валидации как функции оценки.

Тема 2. Скелет ML-пайплайна в scikit-learn

код

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.ensemble import GradientBoostingClassifier

# Пайплайн для числовых признаков: заполнение пропусков + нормализация
num_pipe = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler()) ])

# Пайплайн для категориальных признаков: заполнение + кодирование
cat_pipe = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('encoder', OneHotEncoder(handle_unknown='ignore')) ])

# Объединяем в ColumnTransformer — каждая группа обрабатывается отдельно
prep = ColumnTransformer([
    ('num', num_pipe, num_cols), # числовые столбцы
    ('cat', cat_pipe, cat_cols) # категориальные столбцы
])

# Полный пайплайн: предобработка -> модель
pipe = Pipeline([('prep', prep),
    ('model', GradientBoostingClassifier(n_estimators=200))])
pipe.fit(X_train, y_train) # обучение всего пайплайна
```

Тема 2. Автоопределение типов признаков

код

```
from sklearn.compose import make_column_selector as sel
from sklearn.preprocessing import (
    RobustScaler, PowerTransformer, OneHotEncoder)
from sklearn.feature_selection import SelectKBest, f_classif

# Автоматическое определение типов столбцов по dtype
num_cols = sel(dtype_include='number') # числовые
cat_cols = sel(dtype_include='object') # категориальные

# Расширенный пайплайн с отбором признаков
advanced_pipe = Pipeline([
    ('prep', ColumnTransformer([
        ('num', Pipeline([
            # числовая ветка
            ('imputer', SimpleImputer(strategy='median')),
            ('power', PowerTransformer()), # нормализация
            ('scaler', RobustScaler())    # устойчивость к выбросам
        ]), num_cols),
        ('cat', Pipeline([
            # категориальная ветка
            ('imputer', SimpleImputer(
                strategy='constant', fill_value='missing')),
            ('enc', OneHotEncoder(
                handle_unknown='ignore',
                sparse_output=False))
        ]), cat_cols)
    ])),
    ('select', SelectKBest(f_classif, k=20)), # отбор признаков
    ('model', GradientBoostingClassifier()) # классификатор
])
```

Тема 3. Оптимизация гиперпараметров

ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

Оптимизация гиперпараметров (Hyperparameter Optimization, HPO) является центральным компонентом любой AutoML-системы. Гиперпараметры — это параметры алгоритма, которые не обучаются из данных, а задаются до начала обучения: глубина дерева, скорость обучения, коэффициент регуляризации и т.д.

Grid Search (полный перебор) — наивный подход, при котором оцениваются все комбинации значений из заданной сетки. Вычислительная сложность растёт экспоненциально с числом гиперпараметров (проклятие размерности).

Random Search (случайный поиск) — метод, предложенный Бергстрой и Бенжио (2012), показавший, что случайная выборка из пространства гиперпараметров часто эффективнее полного перебора, поскольку покрывает больше значений «важных» гиперпараметров при фиксированном бюджете.

Байесовская оптимизация — итеративный подход, строящий суррогатную модель (Gaussian Process или TPE) зависимости метрики от гиперпараметров. На каждой итерации выбирается точка, максимизирующая функцию приобретения (acquisition function), что обеспечивает баланс между исследованием новых областей (exploration) и эксплуатацией известных хороших конфигураций (exploitation).

Тема 3.1. Grid Search — полный перебор

код

```
from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import load_breast_cancer

# Загрузка данных для экспериментов
X, y = load_breast_cancer(return_X_y=True)

# Определяем сетку гиперпараметров для перебора
param_grid = {
    'n_estimators': [50, 100, 200], # число деревьев
    'max_depth': [5, 10, 20], # глубина дерева
    'min_samples_split': [2, 5, 10] # мин. объектов для разбиения
}
# Всего: 3 × 3 × 3 = 27 комбинаций × 5-fold CV = 135 обучений

grid = GridSearchCV(
    estimator=RandomForestClassifier(random_state=42),
    param_grid=param_grid,
    cv=5, # 5-fold кросс-валидация
    scoring='f1', # метрика оценки
    n_jobs=-1, # параллельное выполнение
    verbose=1 # прогресс в консоль
)
grid.fit(X, y)

# Лучший результат и параметры
print(f'Grid Search лучший F1: {grid.best_score_:.4f}')
print(f'Лучшие параметры: {grid.best_params_}')
```

Тема 3.2. Random Search — случайный поиск

код

```
from sklearn.model_selection import RandomizedSearchCV
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import load_breast_cancer
from scipy.stats import randint # распределения для выборки

X, y = load_breast_cancer(return_X_y=True)

# Распределения вместо фиксированных значений
param_distributions = {
    'n_estimators': randint(50, 500), # равномерное [50..500)
    'max_depth': randint(2, 30), # равномерное [2..30)
    'min_samples_split': randint(2, 20), # равномерное [2..20)
}

# Случайная выборка 30 конфигураций из пространства
rand = RandomizedSearchCV(
    estimator=RandomForestClassifier(random_state=42),
    param_distributions=param_distributions,
    n_iter=30, # число случайных выборок
    cv=5, # кросс-валидация
    scoring='f1', # метрика
    random_state=42, # воспроизводимость
    n_jobs=-1
)
rand.fit(X, y)

# Сравнение с Grid Search
print(f'Random Search лучший F1: {rand.best_score_:.4f}')
print(f'Лучшие параметры: {rand.best_params_}')
```

Тема 3.3. Bayesian Search — Optuna

код

```
import optuna # фреймворк байесовской оптимизации
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score
from sklearn.datasets import load_breast_cancer

X, y = load_breast_cancer(return_X_y=True)

def objective(trial):
    """Целевая функция для оптимизации Optuna."""
    # suggest_* — определяет пространство поиска
    n_est = trial.suggest_int('n_estimators', 50, 500)
    depth = trial.suggest_int('max_depth', 2, 20)
    min_spl = trial.suggest_int('min_samples_split', 2, 20)
    max_f = trial.suggest_categorical(
        'max_features', ['sqrt', 'log2', None])

    clf = RandomForestClassifier(
        n_estimators=n_est, max_depth=depth,
        min_samples_split=min_spl,
        max_features=max_f, random_state=42)

    # Возвращаем среднее F1 на 5-fold CV
    return cross_val_score(
        clf, X, y, cv=5, scoring='f1').mean()

# TPE-сэмплер автоматически балансирует exploration/exploitation
study = optuna.create_study(direction='maximize')
study.optimize(objective, n_trials=100)
print(f'Bayesian Search лучший F1: {study.best_value:.4f}')
```

Тема 4. TPOT: генетическое программирование

ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

TPOT (Tree-based Pipeline Optimization Tool) — библиотека AutoML, использующая генетическое программирование (GP) для автоматического конструирования ML-пайплайнов. Разработана в Университете Пенсильвании (Olson et al., 2016). TPOT представляет пайплайн как дерево, в котором узлы — операторы scikit-learn: предобработка, отбор признаков, классификатор или регрессор.

Алгоритм работает следующим образом. На первом шаге генерируется начальная популяция случайных пайплайнов (деревьев). Каждый пайплайн оценивается на кросс-валидации. Далее применяются генетические операторы: селекция (турнирная), скрещивание (обмен поддеревьями) и мутация (замена узла). Процесс повторяется заданное число поколений, при этом популяция эволюционирует в направлении более качественных пайплайнов.

Ключевое преимущество TPOT — возможность экспорта найденного пайплайна в воспроизводимый Python-скрипт. Это обеспечивает полную прозрачность: исследователь видит каждый шаг преобразования и может модифицировать его. Кроме того, TPOT поддерживает пользовательские конфигурации пространства поиска, что позволяет ограничить набор рассматриваемых алгоритмов.

Тема 4. TPOT: пример использования

КОД

```
from tpot import TPOTClassifier
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

# Загрузка набора данных Iris (3 класса, 4 признака)
X, y = load_iris(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

# Настройка генетического алгоритма TPOT
tpot = TPOTClassifier(
    generations=5,      # число поколений эволюции
    population_size=20, # размер популяции пайплайнов
    cv=5,               # 5-fold кросс-валидация
    scoring='accuracy', # метрика оценки
    random_state=42,    # воспроизводимость
    verbosity=2,        # подробный лог процесса
    n_jobs=-1           # параллелизация
)
tpot.fit(X_tr, y_tr) # запуск эволюционного поиска

# Оценка на тестовой выборке
print(f'Test accuracy: {tpot.score(X_te, y_te):.4f}')

# Экспорт найденного пайплайна в Python-скрипт
tpot.export('iris_pipeline.py')
print(f'Найденный пайплайн: {tpot.fitted_pipeline_}')
```

Тема 4. Задача А: классификация Wine с TPOT

ПРАКТИЧЕСКАЯ ЗАДАЧА

❑ **Задание:** Обучите TPOT на Wine dataset (3 класса). Оцените ассурасу и экспортируйте пайплайн. Проанализируйте структуру найденного пайплайна.

```
from tpot import TPOTClassifier
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split

# Wine dataset: 13 признаков, 3 класса, 178 объектов
X, y = load_wine(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

# Запуск AutoML с генетическим поиском
tpot = TPOTClassifier(
    generations=5, population_size=20,
    cv=5, scoring='accuracy',
    random_state=42, verbosity=2)
tpot.fit(X_tr, y_tr)

# Результаты
print(f'Test accuracy: {tpot.score(X_te, y_te):.4f}')
print(f'Структура: {tpot.fitted_pipeline_}')
tpot.export('wine_pipeline.py') # сохранение в файл
```

Тема 5. Auto-sklearn: мета-обучение и ансамбли

ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

Auto-sklearn (Feurer et al., 2015, 2020) — AutoML-система, построенная поверх scikit-learn и использующая три ключевых механизма: мета-обучение (meta-learning), байесовскую оптимизацию (SMAC) и автоматическое построение ансамблей.

Мета-обучение позволяет «горячий старт» оптимизации. Система хранит базу мета-признаков (число объектов, признаков, долю пропусков, статистики распределений) для ~140 наборов данных OpenML и соответствующие лучшие конфигурации. При поступлении нового набора данных вычисляются мета-признаки и по ближайшим соседям определяются наиболее перспективные начальные конфигурации.

Байесовская оптимизация SMAC (Sequential Model-based Algorithm Configuration) использует случайный лес как суррогатную модель, что позволяет эффективно обрабатывать условные (conditional) и категориальные гиперпараметры.

Автоматическое ансамблирование — финальный этап: вместо выбора одной лучшей модели Auto-sklearn строит взвешенный ансамбль из всех обученных конфигураций, используя жадный алгоритм отбора с минимизацией ошибки на валидации.

Тема 5. Auto-sklearn: пример использования

код

```
import autosklearn.classification
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.metrics import f1_score, classification_report

# Загрузка данных (диагностика рака молочной железы)
X, y = load_breast_cancer(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Инициализация Auto-sklearn с временным бюджетом
automl = autosklearn.classification\
    .AutoSklearnClassifier(
        time_left_for_this_task=180, # общий бюджет: 3 минуты
        per_run_time_limit=30,      # лимит на одну модель
        ensemble_size=10,           # размер ансамбля
        metric=autosklearn.metrics.f1,
        seed=42)

automl.fit(X_tr, y_tr) # запуск AutoML

# Оценка результатов
y_pred = automl.predict(X_te)
print(f'AutoML F1: {f1_score(y_te, y_pred):.4f}')
print(classification_report(y_te, y_pred))

# Состав итогового ансамбля (какие модели и их веса)
print(automl.show_models())
```

Тема 5. Задача Б: Auto-sklearn на Digits

ПРАКТИЧЕСКАЯ ЗАДАЧА

 **Задание:** На Digits dataset обучите Auto-sklearn (бюджет 120с). Сравните F1-макро с базовым RandomForest. Выведите состав ансамбля.

```
import autosklearn.classification
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.metrics import f1_score
from sklearn.ensemble import RandomForestClassifier

# Digits: 64 признака (пиксели 8×8), 10 классов (цифры 0-9)
X, y = load_digits(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Базовый RF для сравнения
rf = RandomForestClassifier(random_state=42).fit(X_tr, y_tr)
print(f'RF F1-macro: {f1_score(y_te, rf.predict(X_te),
    average="macro"):.4f}')

# Auto-sklearn
aml = autosklearn.classification\
    .AutoSklearnClassifier(
    time_left_for_this_task=120, per_run_time_limit=20,
    seed=42)
aml.fit(X_tr, y_tr)
print(f'AutoML F1-macro: {f1_score(y_te, aml.predict(X_te),
    average="macro"):.4f}')
```

Тема 6. H2O AutoML: масштабируемая платформа

H2O AutoML — компонент распределённой ML-платформы H2O.ai, реализующий автоматический поиск лучшей модели с минимальной конфигурацией. В отличие от TPOT и Auto-sklearn, H2O использует фиксированный набор алгоритмов: GLM, Random Forest, GBM, XGBoost, Deep Learning (многослойный перцептрон), а также строит Stacked Ensemble из лучших обученных моделей.

Архитектура H2O основана на распределённом вычислительном фреймворке, хранящем данные в оперативной памяти в колоночном формате (H2OFrame). Это обеспечивает обработку наборов данных, превышающих объём памяти одной машины. AutoML-модуль последовательно обучает модели в рамках заданного временного бюджета и формирует leaderboard — ранжированный список по метрике.

Stacked Ensemble — ансамбль, в котором предсказания базовых моделей используются как признаки для мета-модели (по умолчанию — GLM с L1-регуляризацией). H2O строит два вида ансамблей: Best-of-Family (по одной лучшей модели каждого типа) и All Models (все обученные модели). Данный подход обеспечивает наивысшее качество за счёт комбинирования разнородных моделей.

Тема 6. H2O AutoML: пример использования

код

```
import h2o
from h2o.automl import H2OAutoML
from sklearn.datasets import fetch_california_housing

# Инициализация JVM-кластера H2O
h2o.init()

# Загрузка данных и конвертация в H2OFrame
data = fetch_california_housing(as_frame=True).frame
hf = h2o.H2OFrame(data)
train, test = hf.split_frame(ratios=[0.8], seed=42)

target = 'MedHouseVal' # целевая переменная
features = [c for c in hf.columns if c != target]

# Запуск AutoML с временным бюджетом
aml = H2OAutoML(
    max_runtime_secs=120, # бюджет 2 минуты
    seed=42,
    sort_metric='RMSE'    # сортировка leaderboard
)
aml.train(x=features, y=target, training_frame=train)

# Результаты: leaderboard и метрики лидера
print(aml.leaderboard.head(10)) # топ-10 моделей
perf = aml.leader.model_performance(test)
print(f'RMSE: {perf.rmse():.4f}, R2: {perf.r2():.4f}')
```

Тема 6. Задача В: регрессия California Housing

ПРАКТИЧЕСКАЯ ЗАДАЧА

📄 **Задание:** Запустите H2O AutoML на 180 секунд. Сравните RMSE лидера и Stacked Ensemble. Выведите важность признаков лидера.

```
import h2o
from h2o.automl import H2OAutoML
from sklearn.datasets import fetch_california_housing

h2o.init() # запуск кластера
data = fetch_california_housing(as_frame=True).frame
hf = h2o.H2OFrame(data)
train, test = hf.split_frame(ratios=[0.8], seed=42)

# Увеличенный бюджет для лучшего результата
aml = H2OAutoML(max_runtime_secs=180, seed=42,
                sort_metric='RMSE')
aml.train(
    x=[c for c in hf.columns if c != 'MedHouseVal'],
    y='MedHouseVal', training_frame=train)

# Анализ leaderboard
print(aml.leaderboard.head(10))
perf = aml.leader.model_performance(test)
print(f'RMSE: {perf.rmse():.4f}, R2: {perf.r2():.4f}')

# Важность признаков лидера
if hasattr(aml.leader, 'varimp'):
    print(aml.leader.varimp(use_pandas=True).head(8))
```

Тема 7. AutoML и несбалансированные данные

ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

Несбалансированность классов — одна из наиболее распространённых проблем при решении задач классификации в реальных приложениях: обнаружение мошенничества, медицинская диагностика, предиктивное обслуживание. Соотношение классов может составлять 1:100 и более, что приводит к смещению модели.

AutoML-системы предоставляют ряд механизмов для работы с дисбалансом. Во-первых, выбор метрики: accuracy неинформативна при дисбалансе, вместо неё используются F1-score, ROC-AUC, Average Precision или log_loss. Во-вторых, взвешивание классов: параметр `class_weight='balanced'` автоматически пересчитывает веса пропорционально обратной частоте классов.

В-третьих, ресемплинг: оверсемплинг миноритарного класса (SMOTE, ADASYN) или андерсемплинг мажоритарного (RandomUnderSampler). Библиотека imbalanced-learn интегрируется с sklearn Pipeline. TPOT поддерживает кастомные операторы ресемплинга. H2O реализует встроенный параметр `balance_classes=True`, выполняющий оверсемплинг автоматически.

Тема 7. Стратегии ресемплинга: SMOTE

КОД

```
from imblearn.over_sampling import SMOTE
from imblearn.pipeline import Pipeline as ImbPipeline
from imblearn.combine import SMOTETomek
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report

# Пайплайн 1: SMOTE + RandomForest
# SMOTE генерирует синтетические объекты миноритарного класса
pipe_smote = ImbPipeline([
    ('smote', SMOTE(random_state=42)), # оверсемплинг
    ('clf', RandomForestClassifier(random_state=42))
])
pipe_smote.fit(X_tr, y_tr)
print('=== SMOTE + RF ===')
print(classification_report(y_te, pipe_smote.predict(X_te)))

# Пайплайн 2: SMOTETomek (оверсемплинг + очистка границ)
# Tomek Links удаляет пограничные объекты после SMOTE
pipe_combo = ImbPipeline([
    ('resample', SMOTETomek(random_state=42)),
    ('clf', RandomForestClassifier(
        class_weight='balanced', # доп. взвешивание
        random_state=42))
])
pipe_combo.fit(X_tr, y_tr)
print('=== SMOTETomek + Balanced RF ===')
print(classification_report(y_te, pipe_combo.predict(X_te)))
```

Тема 7. Задача Г: обнаружение мошенничества

📄 ПРАКТИЧЕСКАЯ ЗАДАЧА

Задание: На синтетическом наборе (5% fraud) обучите TPOT с метрикой `average_precision`. Сравните с моделью без учёта дисбаланса.

```
from sklearn.datasets import make_classification
from tpot import TPOTClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report

# Генерация несбалансированных данных: 95% норма / 5% fraud
X, y = make_classification(
    n_samples=2000, n_features=20,
    weights=[0.95, 0.05], # соотношение классов
    flip_y=0.01,          # шум в метках
    random_state=42)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.3, stratify=y, random_state=42)

# TPOT с метрикой для несбалансированных данных
tpot = TPOTClassifier(
    generations=5, population_size=20,
    scoring='average_precision', # Area Under PR Curve
    cv=5, random_state=42, verbosity=2)
tpot.fit(X_tr, y_tr)

# Отчёт: precision/recall по каждому классу
print(classification_report(y_te, tpot.predict(X_te)))
```

Тема 8. Интерпретируемость моделей AutoML

Автоматизация построения моделей порождает проблему «чёрного ящика»: модель, выбранная AutoML-системой, может быть сложным ансамблем, внутренняя логика которого непрозрачна для исследователя.

Различают два уровня интерпретируемости. Глобальная интерпретация отвечает на вопрос: «Какие признаки в целом наиболее важны для модели?» Для этого используются Permutation Importance и SHAP Summary Plot. Локальная интерпретация объясняет конкретное предсказание: «Почему модель присвоила данному объекту класс 1?» — SHAP Waterfall, LIME.

SHAP (SHapley Additive exPlanations, Lundberg & Lee, 2017) основан на теории кооперативных игр Шепли. Каждому признаку присваивается вклад в отклонение предсказания от среднего. Свойства: аддитивность, локальная точность, согласованность. Для древесных моделей существует эффективный алгоритм TreeSHAP с полиномиальной сложностью.

Практически все AutoML-библиотеки совместимы с SHAP, так как возвращают обученный объект модели, доступный для анализа.

Тема 8. SHAP: глобальная и локальная интерпретация

КОД

```
import shap
import matplotlib.pyplot as plt
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split

# Подготовка данных и обучение модели
X, y = load_breast_cancer(return_X_y=True, as_frame=True)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, random_state=42)
model = GradientBoostingClassifier(
    random_state=42).fit(X_tr, y_tr)

# Создание SHAP-объяснителя (TreeSHAP для древесных моделей)
explainer = shap.Explainer(model, X_tr)
shap_values = explainer(X_te) # вычисление SHAP-значений

# Глобальная интерпретация: какие признаки важнее?
shap.summary_plot(shap_values, X_te, show=False)
plt.tight_layout()
plt.savefig('shap_global.png', dpi=150)

# Локальная интерпретация: почему модель так решила?
shap.plots.waterfall(shap_values[0], show=False)
plt.tight_layout()
plt.savefig('shap_local.png', dpi=150)
```

Тема 8. Задача Д: SHAP-анализ модели TPOT

ПРАКТИЧЕСКАЯ ЗАДАЧА

Задание: Обучите TPOT на Wine dataset. Извлеките финальную модель и постройте SHAP summary plot. Определите три наиболее важных признака.

```
import shap, matplotlib.pyplot as plt
from tpot import TPOTClassifier
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split

# Обучение TPOT на Wine dataset
X, y = load_wine(return_X_y=True, as_frame=True)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, random_state=42)

tpot = TPOTClassifier(
    generations=5, population_size=20,
    cv=5, random_state=42, verbosity=0)
tpot.fit(X_tr, y_tr)

# Извлечение финальной модели из пайплайна TPOT
final_model = tpot.fitted_pipeline_.steps[-1][1]

# SHAP-анализ извлечённой модели
explainer = shap.Explainer(final_model, X_tr)
shap_values = explainer(X_te)
shap.summary_plot(shap_values, X_te, show=False)
plt.tight_layout()
plt.savefig('shap_wine.png', dpi=150)
```

Тема 9. AutoML для задач регрессии

ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

Задачи регрессии в контексте AutoML имеют ряд специфических особенностей. Во-первых, пространство метрик: RMSE, MAE, R^2 , MAPE — каждая метрика предъявляет различные требования к модели. RMSE чувствительна к выбросам, MAE более робастна, R^2 нормализована относительно дисперсии, а MAPE удобна для интерпретации в процентах.

Во-вторых, предобработка целевой переменной: логарифмирование, Box-Cox или Yeo-Johnson трансформации для нормализации распределения. AutoML-системы (H2O, Auto-sklearn) могут включать такие трансформации в пространство поиска.

В-третьих, валидация: для временных рядов стандартная кросс-валидация неприменима, требуется TimeSeriesSplit или expanding window. Optuna позволяет реализовать произвольную схему валидации внутри целевой функции, что даёт максимальную гибкость.

На практике комбинация Optuna + LightGBM/CatBoost является одним из наиболее эффективных подходов для табличных данных регрессии, сочетая скорость обучения с гибкостью настройки.

Тема 9. Optuna + LightGBM: регрессия

код

```
import optuna
import lightgbm as lgb
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import cross_val_score

# Загрузка данных о стоимости жилья в Калифорнии
X, y = fetch_california_housing(return_X_y=True)

def objective(trial):
    """Целевая функция: минимизация RMSE через CV."""
    params = {
        'n_estimators': trial.suggest_int('n', 100, 1000),
        'max_depth': trial.suggest_int('d', 3, 12),
        'learning_rate': trial.suggest_float('lr', .01, .3, log=True),
        'num_leaves': trial.suggest_int('lv', 20, 150),
        'reg_alpha': trial.suggest_float('a', 1e-3, 10, log=True),
        'reg_lambda': trial.suggest_float('l', 1e-3, 10, log=True),
    }
    model = lgb.LGBMRegressor(**params, random_state=42,
                              verbose=-1)
    # neg_RMSE -> инвертируем знак для минимизации
    return -cross_val_score(model, X, y, cv=5,
                            scoring='neg_root_mean_squared_error').mean()

study = optuna.create_study(direction='minimize')
study.optimize(objective, n_trials=80)
print(f'Best RMSE: {study.best_value:.4f}')
```

Тема 9. Задача Е: Optuna + CatBoost

ПРАКТИЧЕСКАЯ ЗАДАЧА

📄 **Задание:** Подберите гиперпараметры CatBoostRegressor для California Housing через Optuna. Сравните RMSE с LightGBM из предыдущего слайда.

```
import optuna
from catboost import CatBoostRegressor
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import cross_val_score

# Те же данные для корректного сравнения
X, y = fetch_california_housing(return_X_y=True)

def objective(trial):
    """Оптимизация CatBoost через Optuna."""
    params = {
        'iterations': trial.suggest_int('it', 100, 800),
        'depth': trial.suggest_int('d', 4, 10),
        'learning_rate': trial.suggest_float('lr', .01, .3, log=True),
        'l2_leaf_reg': trial.suggest_float('l2', 1e-2, 10, log=True),
    }
    model = CatBoostRegressor(**params, random_state=42,
                              verbose=0)
    return -cross_val_score(model, X, y, cv=5,
                            scoring='neg_root_mean_squared_error').mean()

study = optuna.create_study(direction='minimize')
study.optimize(objective, n_trials=60)
print(f'CatBoost best RMSE: {study.best_value:.4f}')
```

Тема 10. Ограничения и лучшие практики AutoML

ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

Несмотря на прогресс, AutoML имеет принципиальные ограничения.

- **Вычислительная стоимость**

Полный перебор пространства пайплайнов требует обучения сотен и тысяч моделей. Для крупных наборов данных (>100 000 объектов) время работы может составлять часы. Стратегии смягчения: ограничение бюджета, ранняя остановка (pruning), подвыборки.

- **Область применимости**

AutoML оптимизирован для табличных данных. Для компьютерного зрения, NLP и временных рядов эффективнее специализированные подходы (NAS, AutoGluon-TimeSeries).

- **Воспроизводимость**

Стохастическая природа поиска означает, что повторные запуски могут давать различные пайплайны. Фиксация `random_state`, логирование в MLflow/W&B и валидация на held-out выборке обязательны.

- **Необходимость экспертного знания**

Требуется знание предметной области для корректной постановки задачи, выбора метрики и валидации результатов. AutoML автоматизирует технические этапы, но не заменяет аналитика.

- **Интерпретируемость**

Сложные ансамбли труднее объяснять заинтересованным сторонам. SHAP и LIME частично решают эту проблему.

Итоги семинара

01

AutoML формализует построение ML-модели как задачу оптимизации (CASH)

02

Grid / Random / Bayesian — три стратегии поиска гиперпараметров

03

TPOT, Auto-sklearn, H2O AutoML, Optuna — ключевые инструменты экосистемы

04

sklearn Pipeline и ColumnTransformer — фундамент любого ML-пайплайна

05

Дисбаланс классов требует специальных метрик и стратегий (SMOTE, SMOTETomek)

06

SHAP обеспечивает глобальную и локальную интерпретируемость AutoML-моделей

07

Optuna + LightGBM/CatBoost — эффективная связка для табличной регрессии

08

AutoML не заменяет эксперта: постановка задачи и валидация за исследователем

Python · TPOT · Auto-sklearn · H2O · Optuna · SHAP · LightGBM · CatBoost